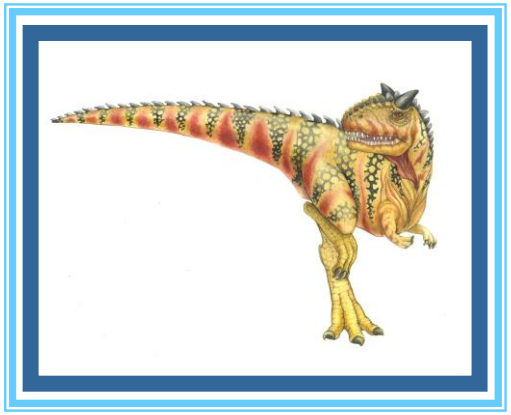
Chapter 8: Deadlocks



Silberschatz, Galvin and Gagne ©2018



Agenda

- Modelo de sistema
- Caracterización del Deadlocks
- Métodos para manejar el Deadlocks
- Prevención de Deadlocks
- Evitar el Deadlocks
- Detección de Deadlocks
- Recuperación de un Deadlocks





Objetivos

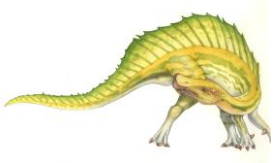
- Ilustrar cómo puede ocurrir un Deadlocks cuando se usan bloqueos mutex
- Definir las cuatro condiciones necesarias que caracterizan el Deadlocks
- Identificar una situación de Deadlocks en un gráfico de asignación de recursos.
- Evaluar los cuatro enfoques diferentes para prevenir Deadlocks
- Aplicar el algoritmo del banquero para evitar Deadlocks
- Aplicar el algoritmo de detección de Deadlocks
- Evaluar los enfoques para recuperarse del Deadlocks





Modelo de Sistema

- El sistema consta de recursos:
- Tipos de Recursos R_1, R_2, . . ., R_m
  - *Ciclos de CPU, espacio de memoria, dispositivos de I/O, entre otros.*
- Cada recurso de tipo R_i tiene W_i instancias.
- Cada proceso utiliza un recurso, considerando el orden siguiente:
  - **Solicitud**
  - **Uso**
  - **Liberación**





Caracterización del Deadlock

El Deadlock puede surgir, si cuatro condiciones se mantienen simultáneamente:

- **Exclusión Mutua:** Sólo un proceso puede usar al mismo tiempo un recurso
- **Retención y Espera:** Un proceso que contiene al menos un recurso, está esperando adquirir recursos adicionales en poder de otros procesos.
- **Sin Prevaciado:** Un recurso puede ser liberado sólo voluntariamente por el proceso que lo posee, después de que ese proceso haya completado su tarea.
- **Espera Circular:** Existe un conjunto $\{P_0, P_1, ..., P_n\}$ de procesos de espera de modo que P_0 está esperando un recurso retenido por P_1, P_1 está esperando un recurso retenido por P_2, ..., P_{n-1} está esperando un recurso retenido por P_n y P_n está esperando un recurso retenido por P_0.





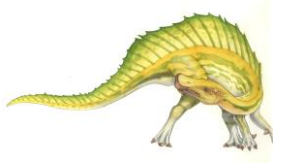
Gráfico de Asignación de Recursos

Sea V, compuesto por un conjunto de vectores de asignación y solicitud

- V se divide en dos tipos:
  - P = {P_1, P_2, ..., P_n}, el conjunto que consta de todos los procesos en el sistema
  - R = {R_1, R_2, ..., R_m}, el conjunto que consta de todos los tipos de recursos en el sistema

- **Vector de Solicitud** – representación del vector: $P_i \rightarrow R_j$ (P_i, R_j)

- **Vector de Asignación** – representación del vector: $R_j \rightarrow P_i$ (R_j, P_i)



Ejemplo de gráfico de Asignación de Recursos

- Una instancia de R1
- Dos instancias de R2
- Una instancia de R3
- Tres instancias de R4
- T1 retiene una instancia de R2 y espera por una instancia de R1
- T2 retiene una instancia de R1, una instancia de R2, y espera por una instancia de R3
- T3 retiene una instancia de R3

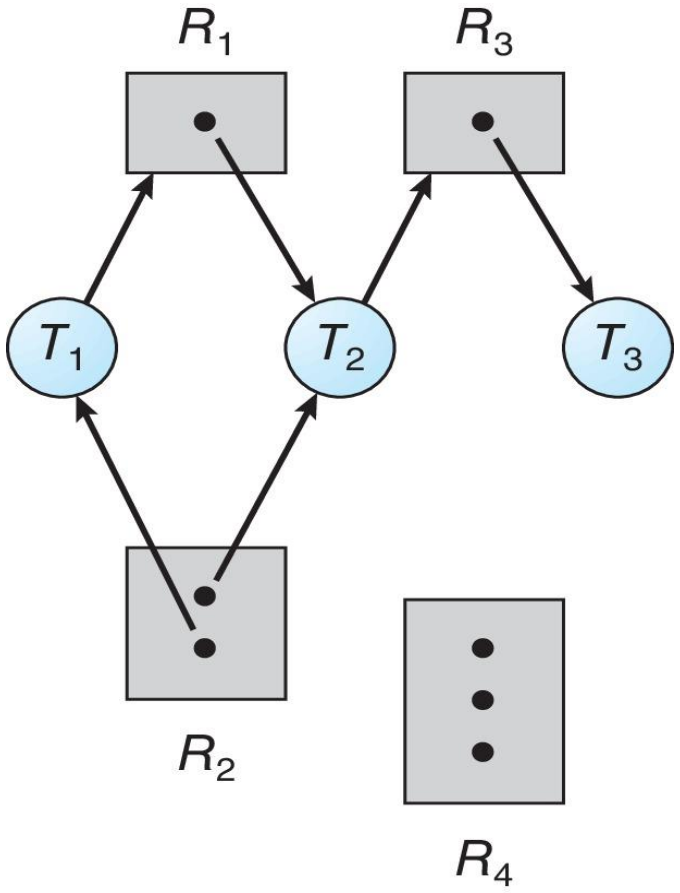


Gráfico de asignación de recursos con Deadlock

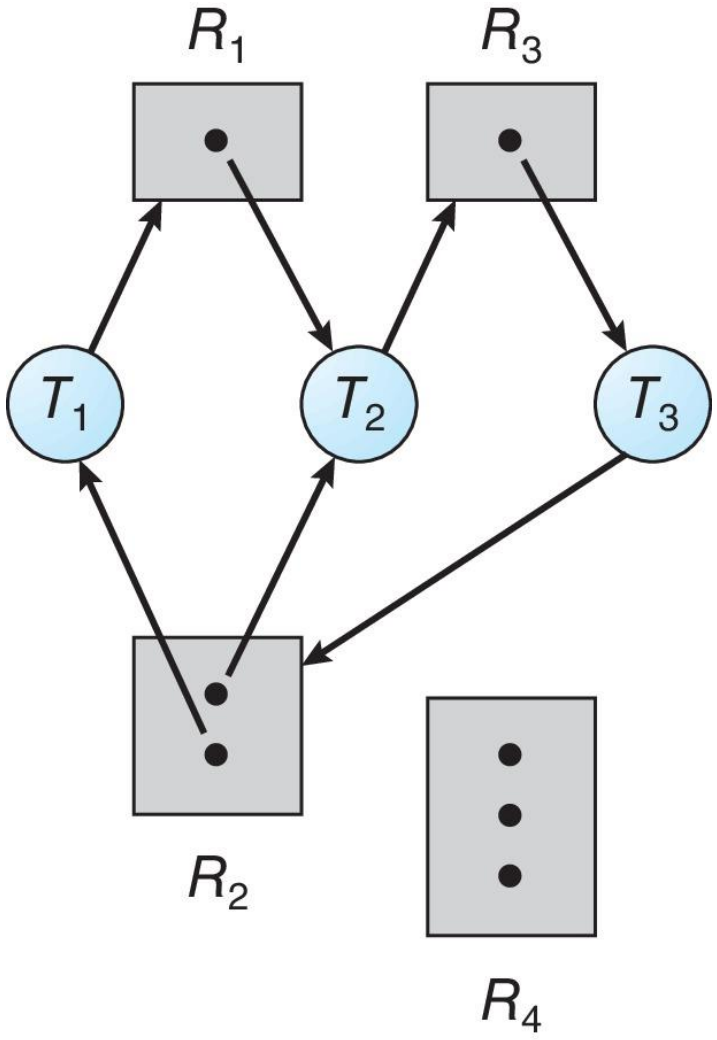
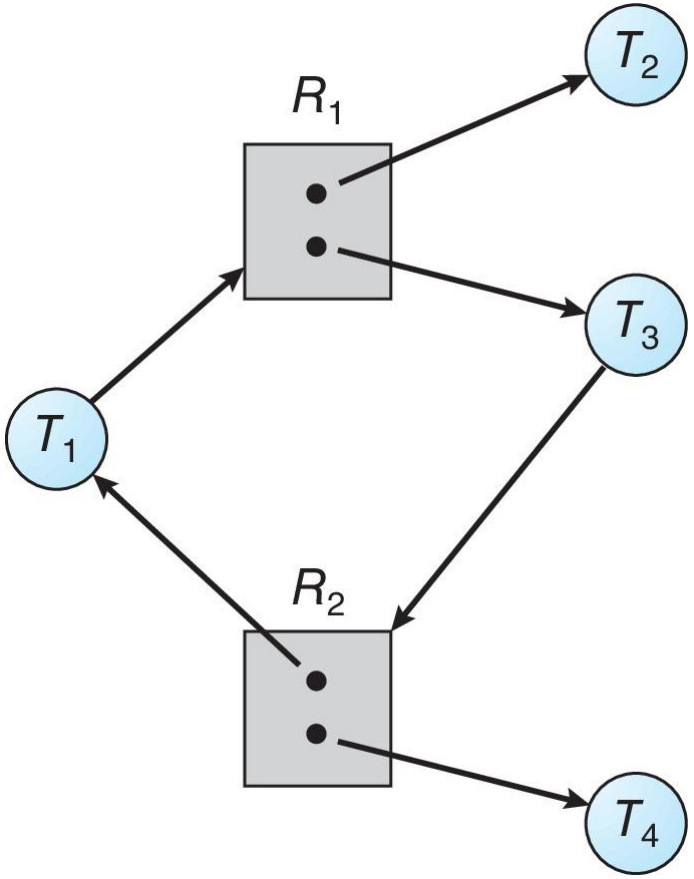


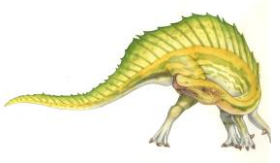
Gráfico con un ciclo pero sin Deadlock





Observaciones

- Si el gráfico no contiene ciclos ⇒ No existe deadlock
- Si el gráfico contiene por lo menos un ciclo ⇒
  - Si solo existe una instancia por tipo de recurso, entonces existe un deadlock
  - Si hay varias instancias por tipo de recurso, existe la posibilidad de un deadlock





Observaciones

- Se define el estado del Sistema como **Estado Seguro**, si el sistema no contiene ciclos (Sin espera circular)

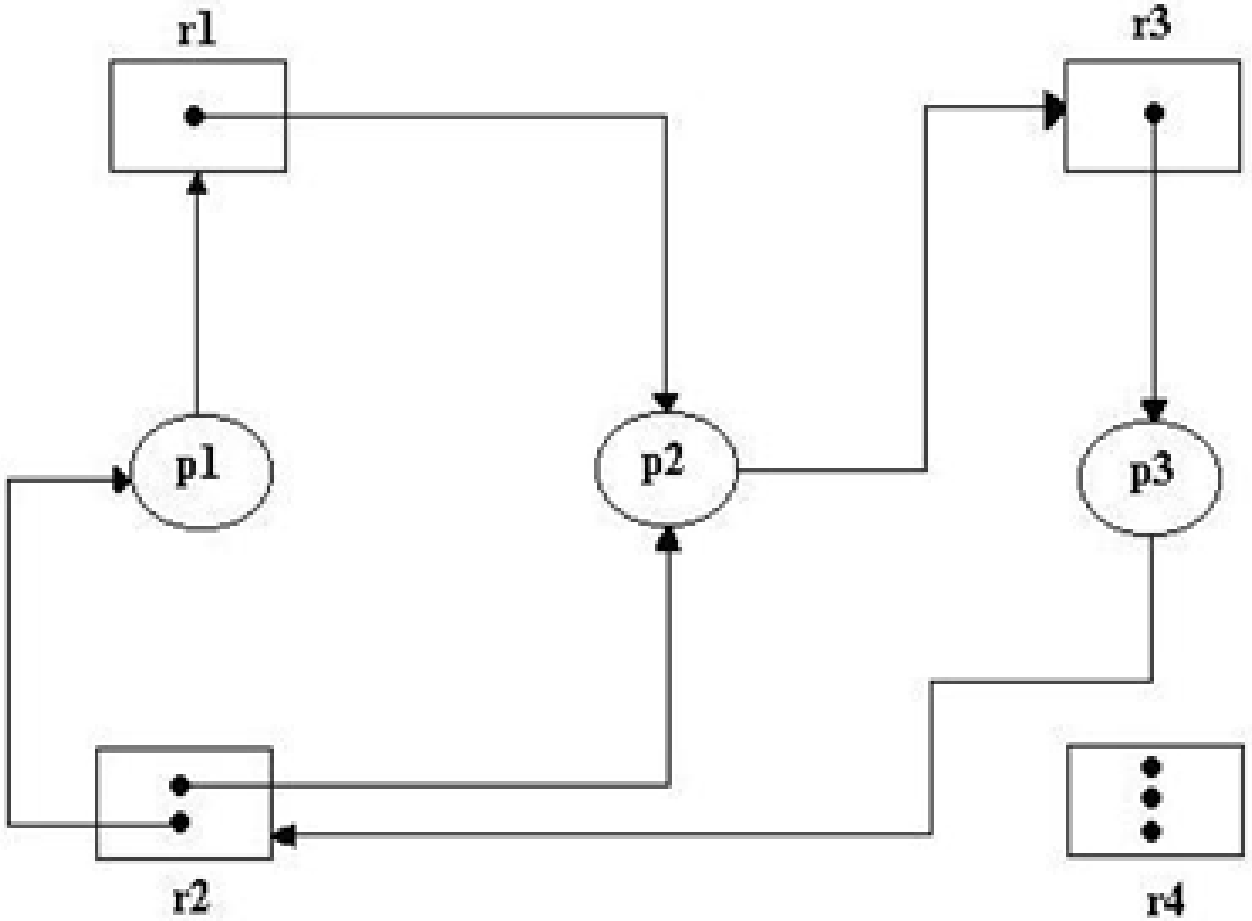
- Se define el estado del Sistema como **Estado Inseguro**, si el sistema contiene por lo menos un ciclo. (Con espera circular)

  - Si solo existe una instancia por tipo de recurso, entonces existe un deadlock, **Estado Inseguro Irreversible**

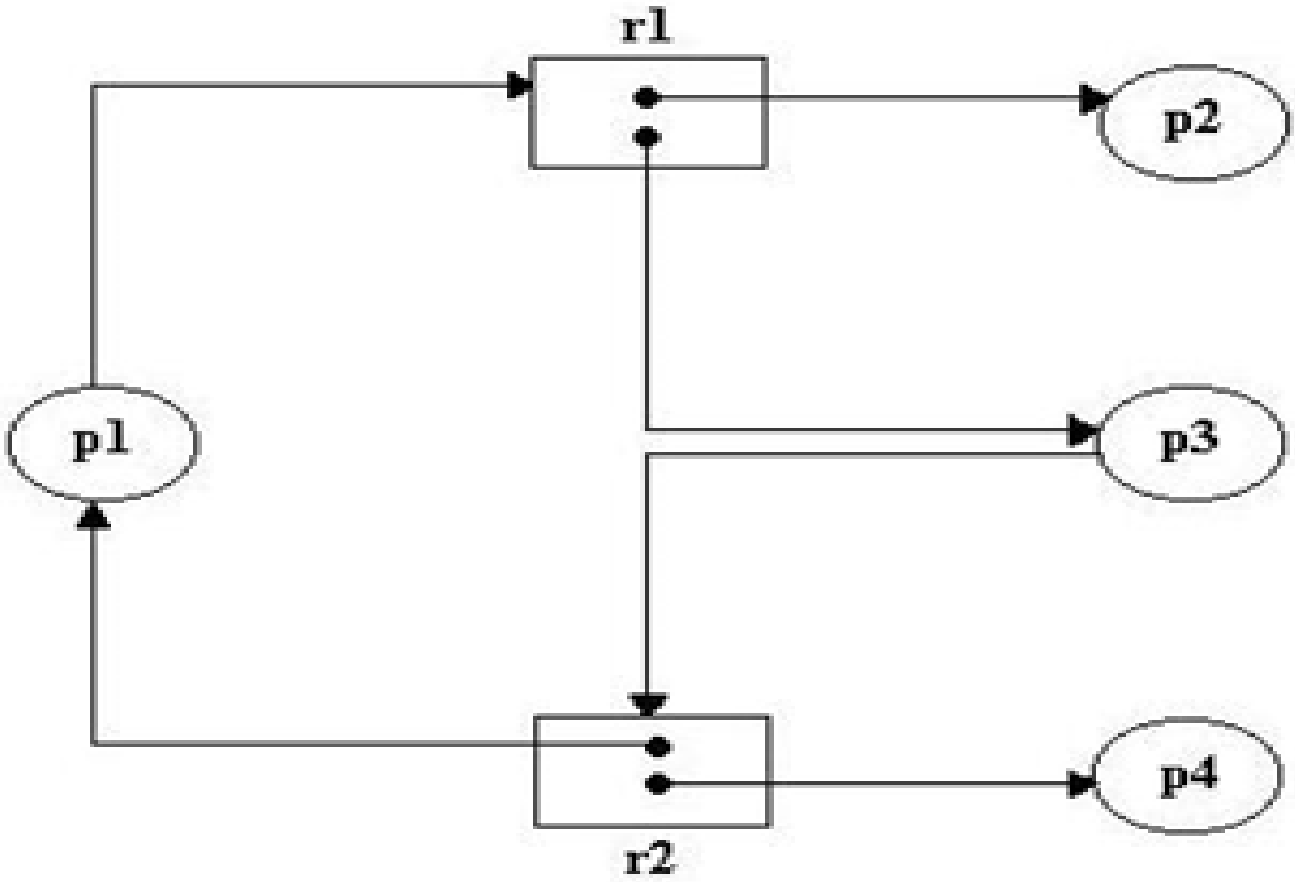
  - Si hay varias instancias por tipo de recurso, existe la posibilidad de un deadlock, **Estado Inseguro Reversible**



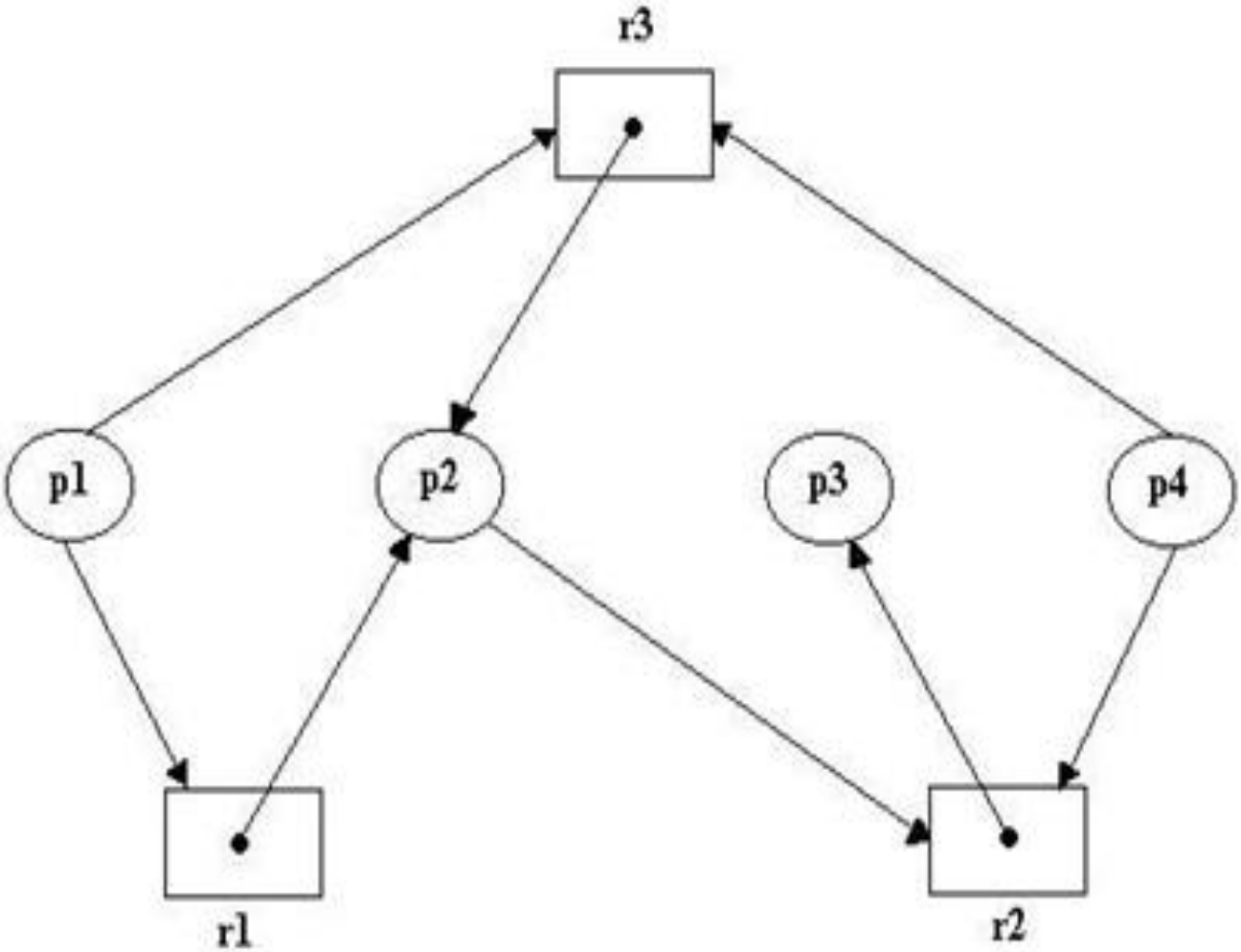
Ejercicio: Determinar el Estado del Sistema



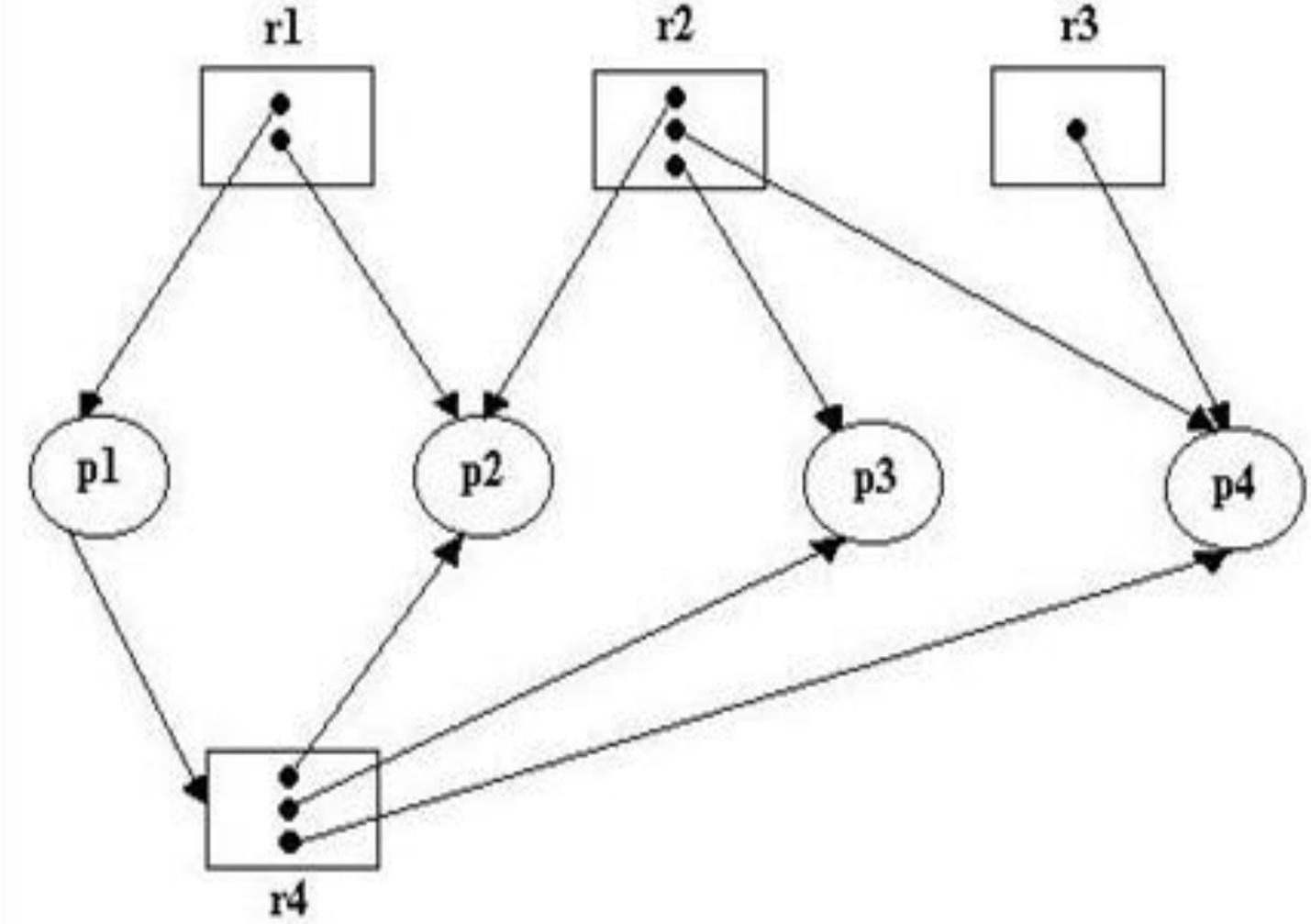
Ejercicio: Determinar el Estado del Sistema



Ejercicio: Determinar el Estado del Sistema



Ejercicio: Determinar el Estado del Sistema





Métodos para manejar el Deadlocks

- Asegurándose de que el sistema nunca ingrese a un estado de deadlock:
  - Prevención del Deadlock.
  - Evitar el Deadlock
- Permitir que el sistema ingrese a un estado de deadlock y recuperar el sistema, eliminando el interbloqueo de procesos.
- Ignorando el problema y pretender que el deadlocks nunca ocurrirá en el Sistema.





Prevención del Deadlock

Invalidando una de las cuatro condiciones necesarias para el deadlock:

- **Exclusión mutua:** no se requiere para recursos compartibles (por ejemplo, archivos de solo lectura); debe mantenerse para recursos no compartibles
- **Retención y Espera:** debe garantizar que cada vez que un proceso solicite un recurso, no contenga ningún otro recurso
  - Requiere que el proceso solicite y se le asignen todos sus recursos antes de que comenzar la ejecución, o permitir que el proceso solicite recursos solo cuando el proceso no tenga ninguno asignado.
  - Baja utilización de recursos; posible starvation





Prevención del Deadlock (Cont.)

- **Sin pre vaciado:**
  - Si un proceso que contiene algunos recursos solicita otro recurso que no se le puede asignar de inmediato, se liberan todos los recursos retenidos actualmente
  - Los recursos previstos se agregan a la lista de recursos que el proceso está esperando
  - El proceso se reiniciará solo cuando pueda recuperar sus recursos antiguos, así como los nuevos que está solicitando.

- **Espera circular:**
  - Imponer un orden total de todos los tipos de recursos y requerir que cada proceso solicite recursos en un orden creciente de enumeración por recurso

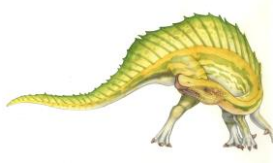




Evitar el Deadlock

Requiere que el sistema tenga alguna información adicional a priori disponible:

- El modelo más simple y útil, requiere que cada proceso declare la cantidad máxima de recursos de cada tipo que pueda necesitar
- El algoritmo de evitar del Deadlock examina dinámicamente el estado de asignación de recursos para garantizar que nunca pueda haber una condición de espera circular
- El estado de asignación de recursos se define por el número de recursos disponibles y asignados, y las demandas máximas de los procesos





Estado Seguro

- Cuando un proceso solicita un recurso disponible, el sistema debe decidir si la asignación inmediata lo deja en un estado seguro

- El sistema está en **estado seguro** si existe una secuencia $<P_1, P_2, ..., P_n>$ de TODOS los procesos en los sistemas, de tal manera, que para cada P_i, los recursos que P_i que aún puede solicitar, pueden ser satisfechos con los recursos disponibles actualmente, + los recursos en poder de todo el P_j, con $j < i$

- Es decir:
  - Si las necesidades de recursos de P_i no están disponibles de inmediato, entonces P_i puede esperar hasta que todos los P_j hayan terminado
  - Cuando P_j finaliza, P_i puede obtener los recursos necesarios, ejecutar, devolver los recursos asignados y finalizar
  - Cuando P_i termina, P_i +1 puede obtener los recursos necesarios, etc.





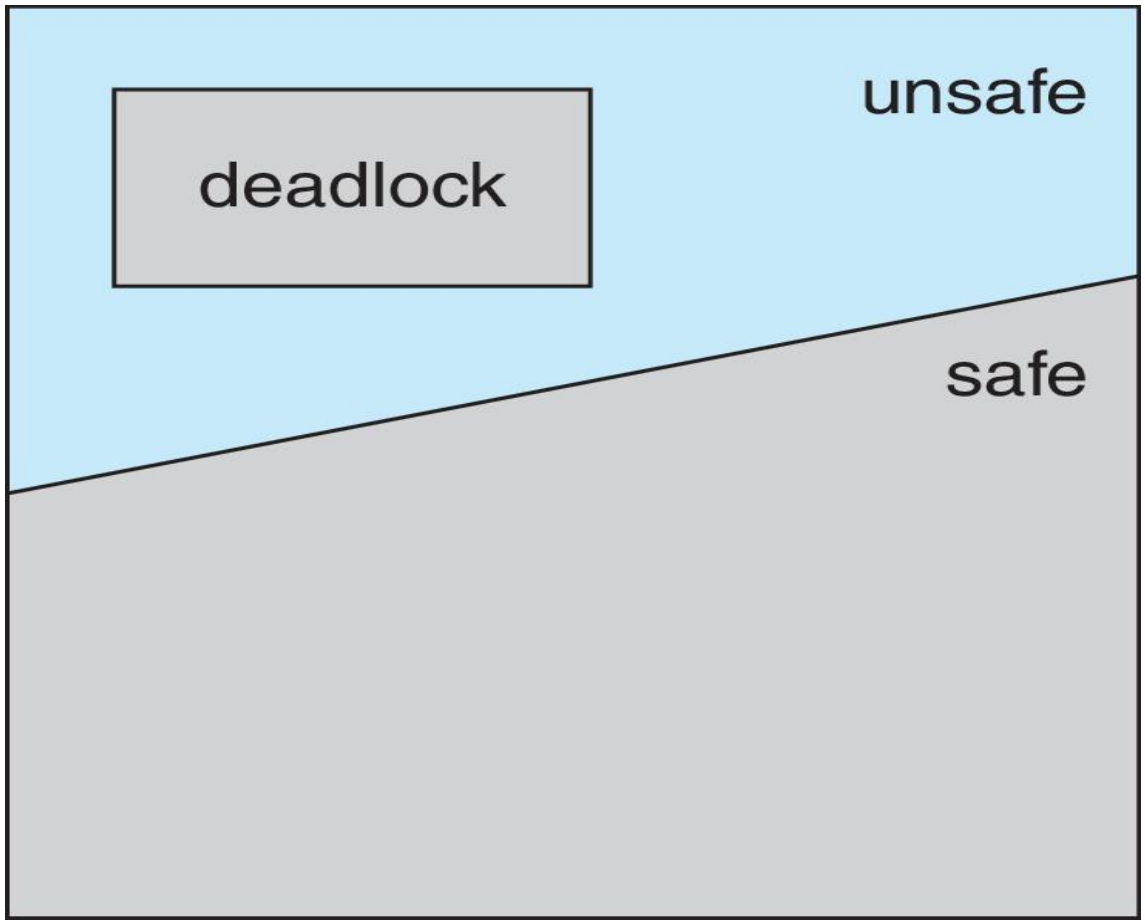
Factores Básicos del Deadlocks

- Si un sistema está en estado seguro => no existe Deadlocks
- Si un sistema está en estado inseguro => Existe la posibilidad de un Deadlocks
- Evitar => Asegurarse de que un sistema nunca entrará en un estado inseguro.





Estado seguro, inseguro, Deadlock







Algoritmos de Evitar

- Instancia única de un tipo de recurso:
  - Usar un gráfico de asignación de recursos
- Múltiples instancias de un tipo de recurso:
  - Usa el **Algoritmo del Banquero**





Algoritmo del Banquero

- Múltiples instancias de recursos.
- Cada proceso debe declarar a priori el uso máximo de cada tipo de recursos.
- Cuando un proceso solicita un recurso, puede ser que tenga que esperar
- Cuando un proceso obtiene todos sus recursos, debe devolverlos en un tiempo finito, cuando termine su ejecución.





Estructura de los Datos, par el Algoritmo del Banquero

Sea n = número de procesos y m = número de tipos de recursos.

- **Disponible:** Vector de longitud m. Si está disponible [j] = k, hay k instancias de tipo de recurso R_j disponibles

- **Máx :** Matriz de n x m . Si Max[i, j] = k, el proceso P_i puede solicitar, como máximo, k instancias del tipo de recurso R_j

- **Asignado:** Matriz de n x m. Si la asignación [i, j] = k, a P_i actualmente se le asignan k instancias de R_j

- **Necesidad:** Matriz de n x m . Si Necesidad[i, j] = k, entonces P_i puede necesitar k más instancias de R_j para completar su tarea

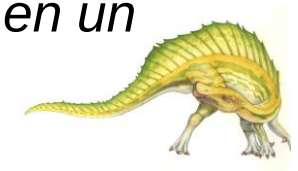
Necesidad[i, j] = Max[i, j] - Asignación[i, j]





Algoritmo de Seguridad

1. Sea Work y Finish vectores de longitud m y n, respectivamente. Inicializar:
   - Work = disponible
   - Finish[i] = falso para i = 0, 1, ..., n- 1
2. Encuentre una i tal que:
   (a) Finish[i] = falso
   (b) Necesidad <=Work
   Si no existe tal i, vaya al paso 4
3. Work = *Work + Asignado*
   Finish[i] = verdadero
   ir al paso 2
4. *Si Finish [i] = verdadero para todo i, entonces el sistema está en un estado seguro*





Algoritmo de solicitud de recursos para el proceso P_i

Solicitud = vector de solicitud para el proceso P_i. Si Solicitud[j] = k, el proceso P_i quiere k instancias del tipo de recurso R_j

1. Si Solicitud <= Necesidad vaya al paso 2. De lo contrario, aumente la condición de error, ya que el proceso ha excedido su reclamo máximo
2. Si Solicitud <= disponible, vaya al paso 3. De lo contrario, P_i debe esperar, ya que los recursos no están disponibles
3. Pretenda asignar los recursos solicitados a P_i modificando el estado de la siguiente manera:

   Disponible = Disponible - Solicitud;

   Asignado = Asignado + Solicitud;

   Necesidad = Necesidad - Solicitud;

   - Si es seguro, los recursos se asignan a P_i
   - Si no es seguro, P_i debe esperar y se restaura el antiguo estado de asignación de recursos.





Ejemplo del Algoritmo del Banquero

- 5 Procesos: P_0 Hasta P_4;

  3 Recursos, de tipo:

  A (10 instancias), *B* (5 instancias), y *C* (7 instancias)

- En el tiempo T_0:

	Asignados	*Max*	*Disponible*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	





Ejemplo (Cont.)

- El contenido de la matriz Necesidad se define como Máx. - Asignación

	Necesidad		
	A	*B*	*C*
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

El sistema se encuentra en un estado seguro ya que la secuencia <P_1, P_3, P_4, P_2, P_0> satisface los criterios de seguridad





Recuperación de un Deadlock: finalización del proceso

- Abortar todos los procesos bloqueados
- Abortar un proceso a la vez hasta que se elimine el ciclo de bloqueo (espera circular)
- ¿En qué orden se debería elegir abortar?
  - Prioridad del proceso.
  - Cuánto tiempo ha calculado el proceso y cuánto tiempo más hasta su finalización
  - Recursos que el proceso ha usado
  - El proceso necesita completarse
  - ¿Cuántos procesos deberán terminarse?
  - ¿El proceso es interactivo o por lotes?

